

InvenTrack

An inventory management app, shipped through a complete DevOps pipeline

Ananya Patel
Enrollment 24bcs10019 · DevOps Final Capstone

github.com/AnanyaPatel3551/inventrack

What InvenTrack does

A web dashboard for tracking products, stock levels and suppliers. Staff add products, update quantities and see stock status at a glance.



Product catalogue
Name, SKU, category, quantity, price, supplier



Stock status
In stock · Low stock · Out of stock



Live KPIs
Totals per status from the /stats endpoint



Responsive UI
Filters, add-product modal, works on mobile

Tech stack

Frontend
React + Vite, Nginx

Backend
FastAPI, SQLAlchemy

Database
PostgreSQL, Alembic

Testing
Pytest

Containers
Docker, Compose

CI/CD
GitHub Actions, GHCR

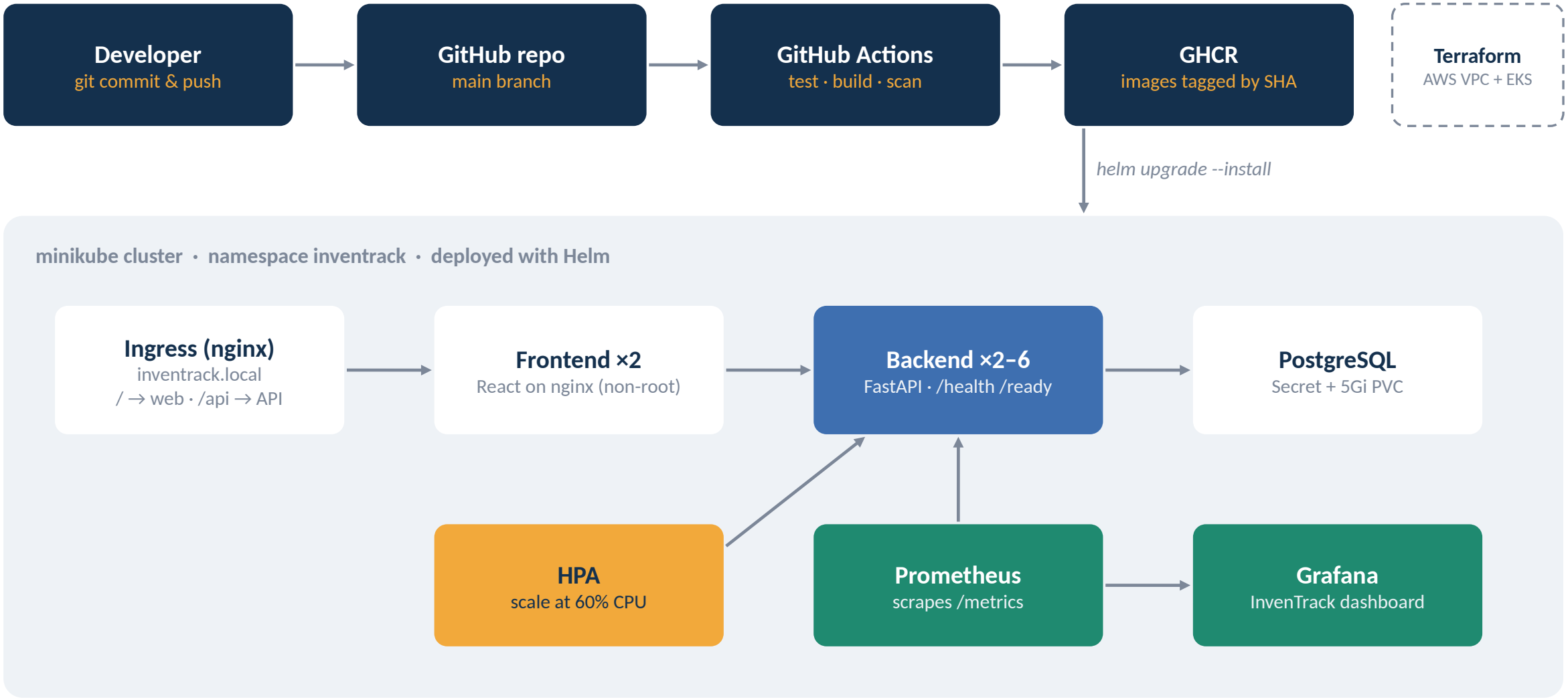
Security
Trivy image scan

Infra as Code
Terraform (AWS)

Orchestration
Kubernetes, Helm

Observability
Prometheus, Grafana

Architecture: from commit to cluster



Application and REST API

Method	Endpoint	Purpose
GET	/api/products	List all products
GET	/api/products/{id}	Get one product
POST	/api/products	Create a product
PUT	/api/products/{id}	Update a product
DELETE	/api/products/{id}	Delete a product
GET	/api/products/stats	Counts per stock status
GET	/health · /ready	Liveness · readiness (checks DB)
GET	/metrics	Prometheus metrics



products table

- id, name, sku (unique)
- category, supplier
- quantity, price
- status, created_at

Schema created by an Alembic migration. Pydantic validates input: an empty name returns 422, a duplicate SKU returns 409.

Testing and containerisation

10

pytest cases, all passing

- Health, root and readiness
- Create, validation (422, 409)
- List, get, 404, update, stats
- Delete and /metrics
- Throwaway SQLite DB via conftest.py, never the real Postgres



Backend image

python:3.12-slim · runs as non-root user (uid 10001) · runs Alembic migrations, then Uvicorn



Frontend image

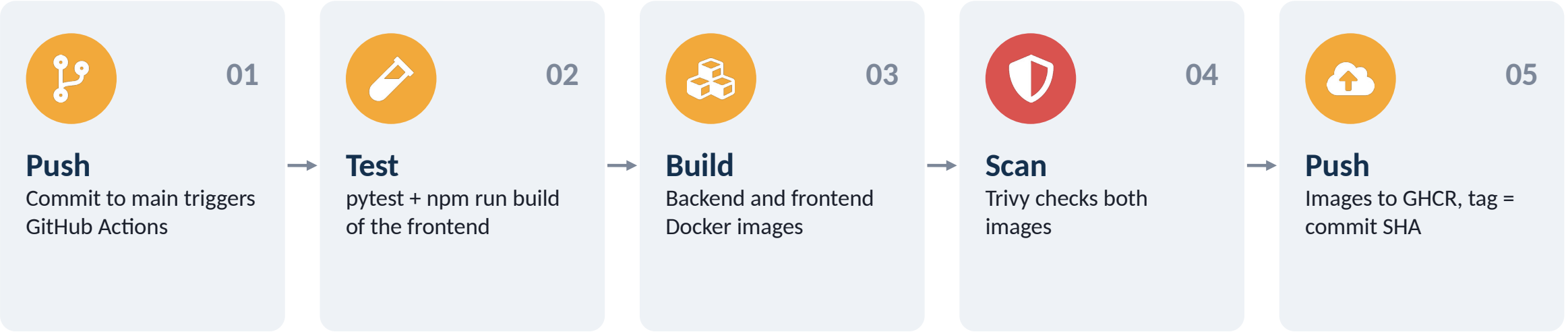
Multi-stage: Node builds the React app → nginx-unprivileged serves it on port 8080



docker-compose.yml

postgres + backend + frontend with one command; healthcheck makes the backend wait for the DB

CI/CD pipeline on every push to main



Fail fast

Jobs run in order. A failing test or a HIGH/CRITICAL CVE with a fix available stops the run, so no broken or vulnerable image reaches the registry.

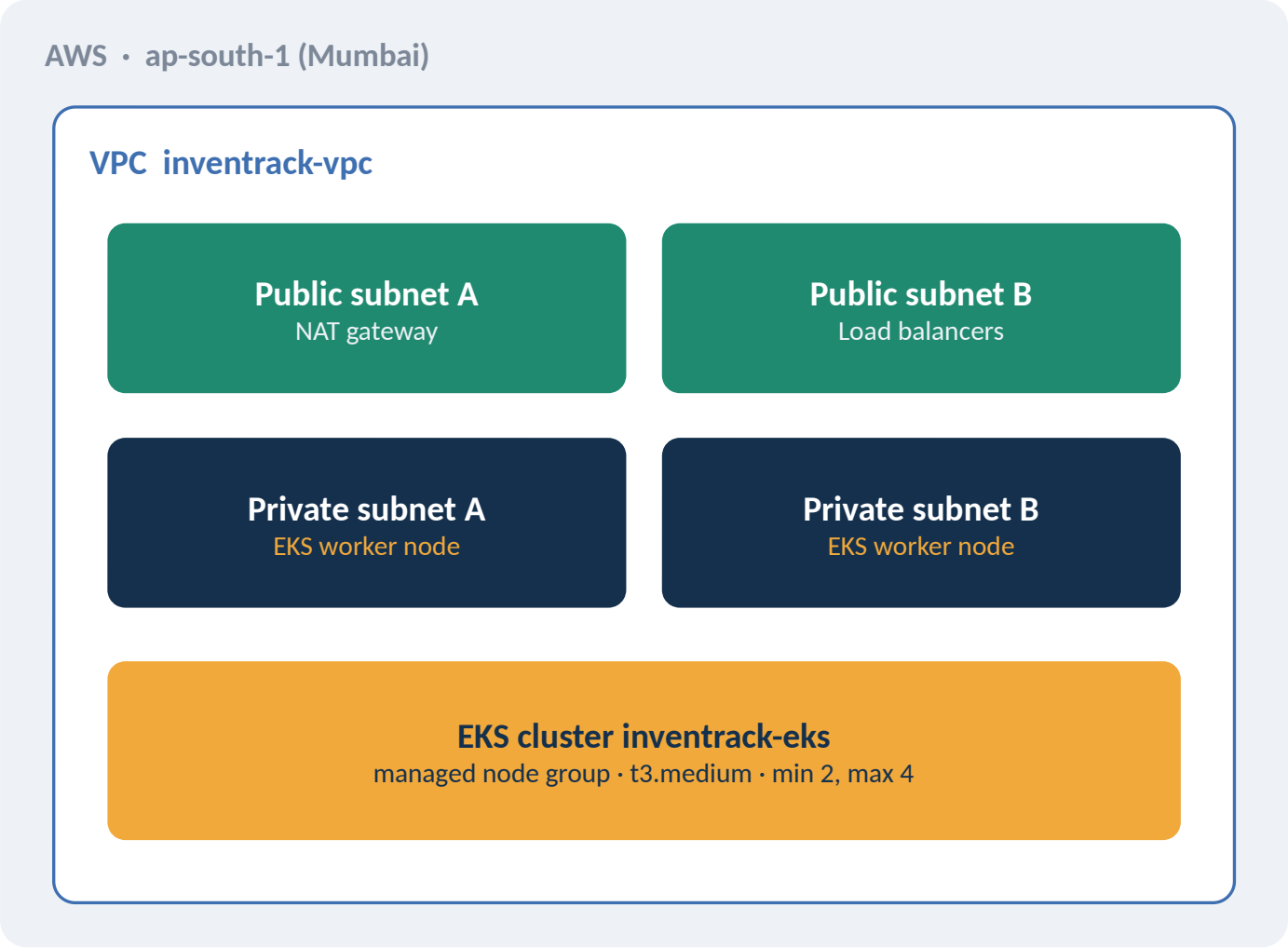
Traceable images

Every image is tagged with the Git commit SHA, never latest. Any running pod can be traced back to the exact commit that built it.

What Trivy checks

OS packages and Python libraries inside each image are matched against CVE databases. A clean result means no fixable HIGH or CRITICAL issues.

Infrastructure as Code with Terraform



Files

- versions.tf, variables.tf, main.tf, outputs.tf
- terraform.tfvars.example (no credentials)
- Official VPC and EKS modules

Status

- terraform init
- terraform fmt -check
- terraform validate
- apply not run: no AWS budget

The app runs on minikube instead; the same Helm chart deploys to EKS unchanged.

Kubernetes deployment with Helm

Deployments
backend ×2, frontend ×2, postgres ×1

Services
ClusterIP for backend, frontend, postgres

Ingress
inventrack.local: / → frontend, /api → backend

HPA
backend 2 → 6 pods at 60% CPU

ConfigMap
BACKEND_URL for the nginx proxy

Secret
DB credentials and DATABASE_URL

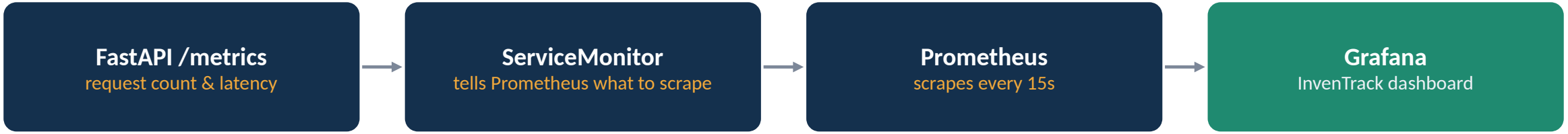
Probes
liveness /health, readiness /ready

Storage
5Gi PVC for PostgreSQL data

One chart, one command

```
helm upgrade --install inventrack ./helm/inventrack -n inventrack -f values-dev.yaml --set
backend.image.tag=<SHA>
```

Monitoring and autoscaling



Dashboard panels

Request rate
per endpoint

p95 latency
per endpoint

5xx error rate
share of requests

Pod count
replicas vs HPA target

Load test → HPA

- busybox pods inside the cluster call /api/products in a loop
- metrics-server reports backend CPU
- Above 60% of the 100m CPU request, the HPA adds pods (max 6)
- Grafana shows traffic and replica count rising

Troubleshooting and key takeaways

ImagePullBackOff

Cause
Image tag doesn't exist in the registry
Find & fix
kubectl describe pod → fix the tag to a real SHA

Service with no endpoints

Cause
Selector labels don't match any pod
Find & fix
kubectl get endpoints → fix the selector

Running but never Ready

Cause
Readiness probe points at a wrong path
Find & fix
kubectl describe → probe /ready

Gates before images
Tests and Trivy run before anything is pushed

SHA tags
Every pod traces back to one commit

Names must line up
Ingress → Service → port → pod labels

Source code github.com/AnanyaPatel3551/inventrack